

DOCUMENTO TÉCNICO DO PRODUTO

MarketplaceCopilot

Plataforma SaaS B2B para vendedores brasileiros de alto volume em Mercado Livre, Shopee e Amazon.

REPOSITÓRIO	FEATURE BRANCH	CONSTITUIÇÃO	DATA
MarketplaceAdvisory.Core	001-marketplacecopilot-platform	v1.1.0	26 · Set · 2026

"Elimine a cegueira financeira. Todo write automático que toca preço protege sua margem."

Sumário

1. Visão geral do produto
 2. Arquitetura & princípios não-negociáveis
 3. Cinco módulos (com prioridade)
 4. Modelo de domínio essencial
 5. Regras do Motor Financeiro
 6. Referência de endpoints
 - Módulo A · Financial Engine
 - Módulo B · AI & Productivity
 - Módulo C · Auto-parts
 - Módulo D · Smart SAC
 - Módulo E · Dynamic Repricer
 7. Erros padronizados (RFC 7807)
 8. Autorização & multi-tenancy
 9. Status de entrega
 10. Próximos passos
-

1 · Visão geral do produto

MarketplaceCopilot é uma plataforma **SaaS B2B** voltada para vendedores brasileiros de alto volume nos três principais marketplaces — **Mercado Livre**, **Shopee** e **Amazon**. A promessa central é acabar com a “cegueira financeira”: cada venda, cada anúncio e cada write automático precisam tornar visível o **lucro líquido real** do vendedor e, sempre que houver troca de preço via automação, **proteger essa margem**.

Missão em uma frase. Guiar o vendedor com decisões auditáveis sobre custo, preço, listagem, atendimento e reprecificação — sem jamais vender abaixo do piso configurado pelo tenant.

Público-alvo

- Vendedores brasileiros de alto volume (SKUs em centenas ou milhares).
- Operações que gerem margem apertada e precisem ser defendidas dinamicamente.
- Vertical especial: **autopeças** (fitment/compatibilidade veicular).

Superfícies de consumo

- **Angular front-end** → passa pelo **BFF** (aggregators por página).
- **Consumidores diretos da Core API** obtêm um **JWT** via `MarketplaceAdvisory.Security`.
- **Jobs internos** (Quartz) rodam repricer, radar de competidores e pipelines de IA.

2 · Arquitetura & princípios não-negociáveis

Clean Architecture + DDD em **.NET 10**, com CQRS-lite sobre EF Core (writes) + Dapper (reads), MediatR nos handlers, multi-tenancy por *global query filters* e RBAC a partir dos JWTs emitidos pelo Security.

Clean Architecture

DDD · aggregates

CQRS-lite

EF Core (writes)

Dapper (reads)

MediatR

FluentValidation

Mapster

ErrorOr

Refit

Quartz.NET

Redis

Meilisearch

Aspire · OTel

Os 8 princípios da constituição (v1.1.0)

#	Princípio	Resumo
I	Clean Architecture + DDD NON-NEGOT	Layers separadas; regras vivem no Domain.
II	CQRS-lite	Commands via EF; Queries via Dapper.
III	Multi-Tenancy by Default NON-NEGOT	Todo aggregate carrega <code>TenantId</code> ; queries só passam por filtros globais.
IV	RBAC via JWT do Security	Papéis <code>User</code> / <code>Manager</code> .
V	Errors as Values	<code>ErrorOr<T></code> em toda API pública.
VI	Layered Testability, Domain Test-First	Testes primeiro para regras puras.
VII	ACL para integrações	Marketplaces atrás de <code>IMarketplaceAdapter</code> .
VIII	Profitability Guardian NON-NEGOT	Nenhum write automático pode vender abaixo do piso do tenant.

Princípio VIII — Profitability Guardian. Todo *command* que mexe em preço obrigatoriamente consulta o `IProfitabilityFloorGuard` . Overrides humanos são possíveis, mas exigem token com ator, motivo e valor de perda aceita — tudo auditável.

3 · Cinco módulos (com prioridade)

P1 · MVP

A · Financial Engine

“Guardião da Lucratividade.” Calcula lucro líquido, margem e semáforo (Green/Yellow/Red) por SKU e marketplace. Define o piso mínimo de preço que todos os writes automáticos consultam.

P2

E · Dynamic Repricer & Competitor Radar

Monitora competidores por SKU e reduz o preço apenas quando *continua acima do piso*. Segura o preço e emite `ProfitabilityFloorHit` quando baixaria abaixo do mínimo.

P3

B · AI Listing Copier · Imagens · Kits

Reescreve título/descrição a partir de URL do concorrente (sem marcas), remove fundo de imagem e monta bundles (peso Σ , dimensão Max/Max/Sum) com preço via simulador.

P4

C · Auto-parts Compatibility

Busca por código de peça (ex.: BOSCH0986AN0510) e retorna lista de veículos/anos/motores com o *catalog id* oficial do marketplace.

P5

D · Smart SAC (Inbox + IA)

Inbox unificado dos marketplaces com resposta rascunhada por IA que *já consulta* a compatibility list do produto. Aprovação em 1 clique.

4 · Modelo de domínio essencial

Entidade	Função
Tenant	Organização vendedora — carrega thresholds Green/Yellow/Floor, seleção de regime tributário (histórico não-sobreposto), SLAs de refresh.
Product (SKU)	Item vendável — CMV, peso, dimensões, preço por marketplace, status de rentabilidade, lista de compatibilidade (autopeças).
MarketplaceFee	Tabela de comissão + taxa fixa por marketplace/categoria/janela efetiva. Aceita override por tenant.
ShippingTier	Custo de frete subsidiado por faixa de peso volumétrico do marketplace.
TaxProfile	Simples Nacional (taxa flat) ou Lucro Presumido (brackets por atividade), pluggable via ITaxRegime .
ProfitabilityCalculation	Registro imutável de cada cálculo — snapshot dos inputs + resultado. Base de auditoria (SC-008).
RepricerDecision	Auditoria imutável de cada rodada do repricer (preço competidor observado, piso aplicado, ação, novo preço).
CompetitorObservation	Snapshot mais recente de cada URL de competidor monitorado.
Bundle	SKU virtual composto por ≥ 2 SKUs concretos (sem aninhamento em v1).
AIJob	Auditoria de cada chamada de IA (kind, prompt, resposta, provider, custo).
ListingDraft	Saída da IA para reescrita de anúncio (título ≤ 60 chars, sem marcas do concorrente).
ListingCompatibility	Ligação Product → catálogo de compatibilidade do marketplace (autopeças).
CustomerMessage · MessageThread	Item do inbox unificado + agrupamento por thread por marketplace.
MessageReplyDraft	Rascunho de resposta gerado por IA aguardando aprovação humana.

Value objects centrais

Money · TenantId · Weight (g) · Dimensions (cm) · Percentage [-100..100] · ProfitabilityThresholds (Green≥Yellow≥Floor≥0) · HumanOverrideToken · FloorDecision (Allowed | HeldFloorHit | RequiresOverride)

5 · Regras do Motor Financeiro

RULE A1

Lucro líquido

NetProfit = **SalePrice** – **CMV** – (**SalePrice** × **Commission%**) – **FixedUnitFee** – **SubsidizedShipping** – (**SalePrice** × **TaxRate%**)

Cada input é por tenant/SKU/marketplace quando aplicável.

RULE A2

Semáforo

Status = Green se margem ≥ Green threshold · Yellow se ≥ Yellow · Red caso contrário (inclui margens negativas).

Defaults de plataforma: Green ≥ 12%, Yellow ∈ [6%, 12%), Red < 6%.

RULE A3

Simulador de preço ideal

P = (**CMV** + **FixedFee** + **Shipping**) / (1 – **target** – **commission** – **tax**)

Reversível ±0,05 pp; iterador respeita mudança de faixa de frete.

RULE A4

Piso mínimo de preço

Floor = (**CMV** + **FixedFee** + **Shipping**) / (1 – **minFloor** – **commission** – **tax**)

Consumido pelo `IProfitabilityFloorGuard` — todo write automático passa por aqui.

6 · Referência de endpoints

Todos os endpoints:

- Estão versionados sob `/api/v1/...`.
- Exigem **JWT válido** emitido pelo `MarketplaceAdvisory.Security` (Princípio IV).
- Respondem erros com **ProblemDetails (RFC 7807)** via `ToProblem`.
- São escopados por `TenantId` extraído do token (Princípio III).

6.A · Módulo A — Financial Engine `/api/v1/financial`

POST `/products/{productId}/costs`

Manager

Registrar/atualizar custos. Grava CMV, categoria, peso, dimensões e categorias por marketplace. Dispara recomputação imediata do lucro e do semáforo para cada marketplace configurado.

Erros: 400 (CMV/peso negativo), 404 (produto), 409 (concorrência).

POST `/products/{productId}/sale-price`

Manager

Definir preço de venda. Consulta o `IProfitabilityFloorGuard` antes de persistir. Retorna 409 `floor.hit` se o preço fica abaixo do piso, ou 409 `floor.override_required` se o cliente precisa reenviar com `HumanOverrideToken`.

POST `/simulator`

User/Manager

Ideal-Price Simulator (Rule A3). Recebe uma margem-alvo e devolve o preço sugerido que, ao ser recalculado, entrega essa margem $\pm 0,05$ pp — honrando a faixa de frete no preço sugerido.

Erros: 400 (margem fora do range), 422 (nenhuma faixa de frete cobre o peso volumétrico).

GET `/products/{productId}/profitability`

User/Manager

Rentabilidade atual do SKU. Retorna `ProductProfitabilityDto[]` — um por marketplace configurado.

GET `/dashboard`

User/Manager

Dashboard semáforo. Listagem paginada de SKUs e status atual — servida pela view `Dapper catalog.product_profitability_v`. Filtros: `status`, `marketplace`, `page`, `pageSize` (≤ 100).

PUT `/tenants/{tenantId}/thresholds`

Manager

Atualizar limiares do tenant. Green/Yellow/Floor — recomputa o read model do dashboard. Retorna 400 se `green < yellow`, `yellow < floor` ou qualquer valor negativo.

6.E · Módulo E — Dynamic Repricer /api/v1/pricing

POST /repricer/{productId}

Manager

Configurar o repricer para um SKU/marketplace. Liga/desliga, define `underCutDelta` (ex.: 0,01) e `minCadenceMinutes`.

Erros: 400 (delta negativo, cadência abaixo do mínimo do tenant), 404.

GET /repricer/{productId}/history

User/Manager

Histórico das decisões do repricer. Servido por `pricing.repricer_history_v`. Filtros: `marketplace`, `from` / `to` (ISO), `action` ∈ {Lowered, Held, NoChange}, paginação.

POST /radar/{productId}/competitors

Manager

Registrar URL de competidor. A URL vai para o Competitor Radar. Retorno inicial vem com `buyBoxPositionHint` = "Unknown" até a primeira varredura.

Erros: 400 (URL inválida ou duplicada para o SKU).

GET /radar/{productId}/competitors

User/Manager

Listar snapshots atuais de `CompetitorObservation` para o SKU.

Jobs internos (não expostos por HTTP). `RepricerJob` chama `RunRepricerDecisionCommand`, que consulta o guard e grava a `RepricerDecision`. `CompetitorRadarJob` renova as observações via adapter — falha isolada por marketplace.

6.B · Módulo B — AI, Imagens & Kits /api/v1/listings

POST /ai/rewrite

Manager

Reescrita de anúncio a partir de URL. Materializa um `AIJob` (auditoria antes de qualquer coisa) e enfileira o job. Retorno assíncrono 202 com `aiJobId`.

Erros: 400 (URL inválida), 429 (quota de IA do tenant excedida).

GET /ai/jobs/{jobId}

User/Manager

Polling de status do job. Ao sucesso traz `listingDraftId`, título reescrito (≤ 60 chars) e descrição limpa (sem marcas de competidor).

POST /ai/drafts/{draftId}/approve

Manager

Aprovar draft e vinculá-lo ao `Product`. Se o draft trouxer preço sugerido e `keepOriginalPrice=false`, o guard é consultado — 409 `floor.hit` se violar.

POST `/images/clean`

Manager

Limpeza de fundo da imagem. Envia a referência (ex.: `s3://...`), roda pipeline assíncrono (fundo → `#FFFFFF`). Preserva a imagem original. Retorno 202.

POST `/bundles`

Manager

Criar bundle de 2+ SKUs concretos. Calcula peso Σ , dimensões Max/Max/Sum, e chama o simulador para sugerir preço com margem-alvo.

Erros: 400 (< 2 itens, bundle aninhado), 409 (SKU já existe), 404 (item desconhecido).

GET `/bundles/{bundleId}`

User/Manager

Ver bundle com métricas calculadas e projeções de rentabilidade por marketplace.

6.C · Módulo C — Auto-parts `/api/v1/autoparts`

GET `/search`

User/Manager

Busca por código de peça (ex.: `BOSCH0986AN0510`). Retorna veículos/anos/motores com *catalog id* oficial do marketplace. Filtros: `marketplace`, paginação (max 50/pg).

Erros: 400 (partCode vazio), 503 (índice indisponível — degradação graciosa).

POST `/products/{productId}/compatibility`

Manager

Anexar compatibilidades ao SKU. Cada entrada exige `catalogId` não-vazio (FR-C2) — texto livre é rejeitado.

GET `/products/{productId}/compatibility`

User/Manager

Listar compatibilidades já anexadas ao produto.

POST `/products/{productId}/compatibility/publish`

Manager

Publicar no marketplace via adapter, usando exatamente os catalog ids armazenados. Resposta 202 com status por marketplace (503 isolado se um adapter cair).

6.D · Módulo D — Smart SAC `/api/v1/sac`

GET `/inbox`

User/Manager

Inbox unificado escopado por tenant — servido por `support.inbox_v`. Filtros: `status` ∈ {Unread, Read, Answered}, `marketplace`, paginação (max 100).

GET

/threads/{threadId}

User/Manager

Thread completa — todas as CustomerMessage + MessageReplyDraft ligado (se houver).

POST

/threads/{threadId}/draft

Manager

Forçar geração de rascunho pela IA (também disparado automaticamente ao chegar nova inbound). Enfileira um AIJob tipo SacDraft .

Erros: 409 (já existe draft para a última inbound).

POST

/threads/{threadId}/reply/approve

Manager

Aprovar em 1 clique. Envia via adapter e marca a thread como Answered . Aceita editedBody opcional (edição é logada). Se o adapter cai → 503 e a resposta fica em Approved aguardando retry.

POST

/threads/{threadId}/reply/reject

Manager

Rejeitar rascunho. Operador escreve do zero ou gera novo draft.

7 · Erros padronizados (RFC 7807)

Código HTTP	Situação típica
400 Validation	Payload inválido — validador FluentValidation retorna a chave do campo.
401 / 403	JWT ausente/expirado ou papel insuficiente.
404 NotFound	Entidade não encontrada no tenant do request.
409 Conflict — floor.hit	Write bloqueado pelo Profitability Floor Guard.
409 Conflict — floor.override_required	Reenvie com HumanOverrideToken .
422 Unprocessable	Regra de negócio impossível (ex.: sem faixa de frete cobrindo o peso).
429	Quota (IA ou repricer) do tenant excedida.
503	Adapter externo caído — resposta por marketplace, sem cross-failure.

8 · Autorização & multi-tenancy

- Roles: User (leitura + operações leves) e Manager (writes financeiros, config, aprovações).

- **Multi-tenancy (Princípio III):** todo aggregate carrega `TenantId` ; queries só passam por *global query filters*; nenhuma leitura entre tenants sem operação admin explícita.
- **Auditoria:** cada write registra actor, tenant, timestamp, before-state e after-state.
- **Segredos:** nunca no código — todos vêm do provedor de configuração em runtime.

9 · Status de entrega

Fase	Escopo	Status
Planejamento (SpecKit)	Constituição v1.1.0, spec, plan, research, data-model, 5 contracts, tasks T001–T056, quickstart.	✅ Completo
Fase 1 — Domain (núcleo)	VOs (Weight, Dimensions, Percentage, Thresholds, HumanOverrideToken), enums (Status/Trigger/Regime), aggregates Tenant, MarketplaceFee, ShippingTier, ProfitabilityCalculation, Product estendido, eventos.	✅ Entregue
Fase 2 — Financial Engine	<code>ProfitabilityCalculatorService</code> (A1/A2/A4), <code>IdealPriceSimulator</code> (A3), <code>FloorDecision</code> , <code>IProfitabilityFloorGuard</code> + implementação.	✅ Entregue
Fase 1 — Domain (restante)	Bundle, TaxProfile isolado, módulos Pricing/Content/Support/Autoparts.	🕒 Próxima rodada
Fase 2 — Application/CQRS	Commands (SetSalePrice, UpdateProductCosts, SetTenantThresholds), queries, validators, mappings.	🕒 Próxima rodada
Fase 3 — EF + Dapper	Schema, migrations, views, repositórios completos.	🔒 Depende de conn string Postgres
Fase 4 — IA + Jobs	Quartz jobs (Repricer, Radar, ListingRewrite, ImageCleanup, SacDraft), fakes + provider concreto.	🔒 Depende de chaves (AI, storage, marketplaces)
Fase 5 — Controllers + BFF	Todos os endpoints deste documento + BFF aggregators.	🕒 Após 3 e 4

Baseline técnico: `dotnet build + dotnet test` na solução `MarketplaceAdvisory.Core.sln` — **30 / 30 testes verdes** no commit `43e0394` .

10 · Próximos passos

1. **Confirmar decisões** tomadas em autopilot (Percentage assinado, Product v1 com preço único no EF).

2. **Continuar Fase 1/2** com Bundle, Application commands (`SetSalePriceCommand` e amigos) + queries + validators + Mapster.
3. **Fornecer credenciais** para destravar Fases 3 e 4 (Postgres, AI, storage, marketplaces).
4. **Inicializar SpecKit** nos repositórios `BFF` e `Security` para manter a governança consistente.

Este documento é a leitura executiva do que está em `specs/001-marketplacecopilot-platform/`. Para o texto normativo, consulte `spec.md`, `plan.md`, `data-model.md` e cada arquivo em `contracts/`.